AI VIET NAM
@aivietnam.edu.vn

AI VIET NAM – AI COURSE 2026

Step-by-Step: Logistic Regression

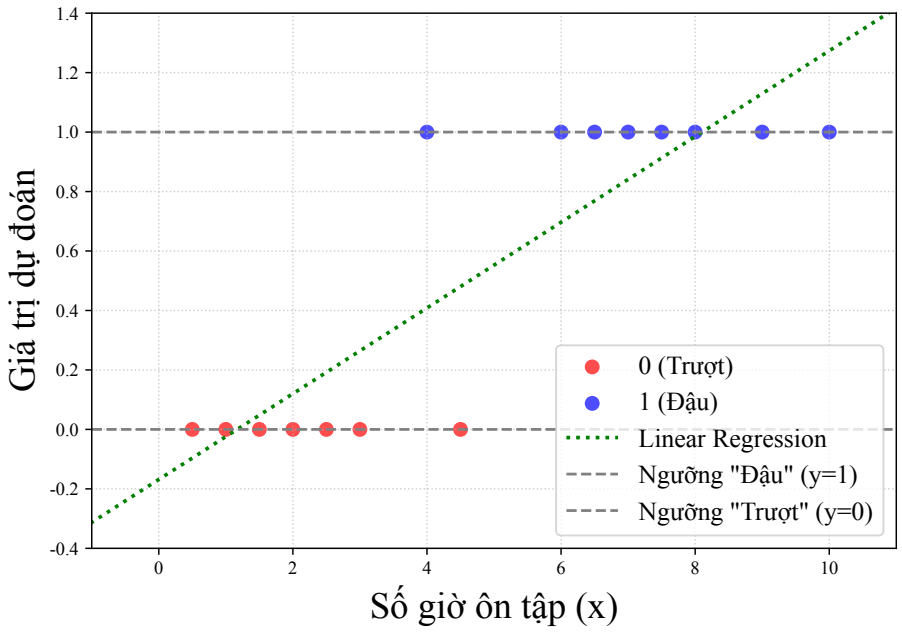
Dương Trường Bình Dương Đình Thắng Đinh Quang Vinh

I. Dẫn nhập

Ở bài học trước về *Simple Linear Regression*, chúng ta đã tìm hiểu cách dự đoán một giá trị liên tục như tiền lương dựa trên một đầu vào như số năm kinh nghiệm. Đường thẳng $f(x) = wx + b$ hoạt động tốt vì đầu ra y có thể nhận bất kỳ giá trị số thực nào. Tuy nhiên, trong thực tế, có nhiều bài toán mà đầu ra không phải là một con số liên tục, mà là một sự lựa chọn giữa hai hoặc nhiều khả năng. Ví dụ:

- Dự đoán một email là spam hay không spam? (0 hoặc 1)
- Dự đoán một khối u là lành tính hay ác tính? (0 hoặc 1)
- Dự đoán một sinh viên thi Đậu hay thi trượt dựa trên số giờ ôn tập? (1 hoặc 0)

Đây được gọi là các bài toán phân loại nhị phân hay Binary Classification.



Hình 1: Đường thẳng Linear Regression dự đoán vượt ra ngoài các giá trị có ý nghĩa [0, 1].

Mục lục

I. Dẫn nhập **1**

II. Thuật ngữ và ký hiệu **4**

III. Từ Linear Regression đến Logistic Regression **6**

III.1. Ôn tập nhanh Linear Regression 6

III.2. Hàm Sigmoid 6

III.3. Hàm mất mát mới: Binary Cross-Entropy 9

III.4. Gradient Descent cho Logistic Regression 12

III.5. Quy trình huấn luyện 15

IV. Thực hành **17**

IV.1. Phát biểu bài toán 17

IV.2. Chi tiết tính toán 17

IV.3. Phân tích kết quả epoch 1 19

IV.4. Phân tích kết quả huấn luyện qua các epoch 20

V. Câu hỏi trắc nghiệm **22**

VI. Tài liệu tham khảo **24**

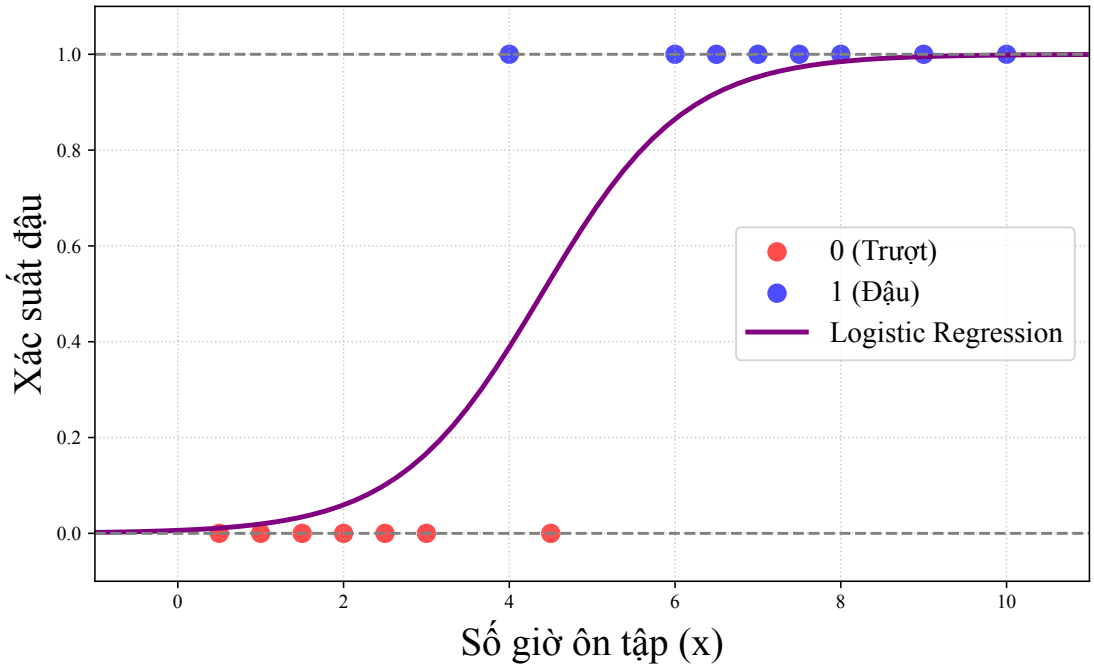
Phụ lục **25**

Hãy xét ví dụ về sinh viên: trục hoành x là số giờ ôn tập, trục tung y chỉ nhận hai giá trị 0 (trượt) và 1 (Đậu). Nếu ta cố gắng dùng Linear Regression để khớp một đường thẳng $f(x) = wx + b$ với dữ liệu này, ta sẽ gặp các vấn đề:

1. Đường thẳng có thể dự đoán các giá trị lớn hơn 1 (ví dụ: $f(x) = 1.5$, nghĩa là "Đậu 150%"?) hoặc nhỏ hơn 0 (ví dụ: $f(x) = -0.3$, nghĩa là "trượt -30%"?). Những con số này dường như không có ý nghĩa thực tế với bài toán.

2. Chúng ta mong muốn đầu ra là một xác suất, ví dụ như $P(\text{Đậu}|x \text{ giờ học})$, và xác suất về bản chất phải nằm trong đoạn $[0, 1]$. Tuy nhiên, đầu ra của Linear Regression là một số thực bất kỳ, có thể vượt ra ngoài đoạn này (như 1.5 hoặc -0.3), do đó nó không đáp ứng được mong muốn này.

Để giải quyết vấn đề này, chúng ta cần một mô hình mới có khả năng nhận đầu ra của hàm tuyến tính (vốn là một số thực bất kỳ từ $-\infty$ đến $+\infty$) và biến đổi nó lại một cách mượt mà vào khoảng $(0, 1)$.



Hình 2: Logistic Regression (màu tím) sử dụng hàm Sigmoid để biến đổi kết quả dự đoán thành một xác suất, luôn nằm trong khoảng (0, 1).

Đây chính là lúc Logistic Regression xuất hiện. Nó sử dụng một hàm đặc biệt gọi là hàm Sigmoid để biến đổi giá trị dự đoán, khiến nó trở thành một công cụ phân loại mạnh mẽ có thể diễn giải được dưới dạng xác suất.

Trong bài viết này, chúng ta sẽ tìm hiểu chi tiết về cách Logistic Regression hoạt động, hàm Sigmoid là gì, và cách chúng ta xây dựng cũng như huấn luyện mô hình này để giải quyết các bài toán phân loại nhị phân.

II. Thuật ngữ và ký hiệu

Trước hết, chúng ta có thể đọc qua một số thuật ngữ và ký hiệu sẽ được dùng trong bài viết.

Bảng thuật ngữ

Thuật ngữ	Ý nghĩa
Phân loại nhị phân	Một bài toán học máy nhằm dự đoán đầu ra thuộc về một trong hai nhóm, thường được gán nhãn là 0 hoặc 1.
Logistic Regression	Mô hình học máy dùng cho bài toán phân loại nhị phân. Mô hình này dự đoán xác suất một đối tượng thuộc về lớp 1.
Hàm Sigmoid (Hàm Logistic)	Một hàm số có đồ thị hình chữ S dùng để "nén" một số thực bất kỳ, ví dụ như đầu ra của Linear Regression, vào khoảng từ 0 đến 1.
Loss Function	Một hàm số đo lường sự khác biệt, hay "lỗi", giữa giá trị dự đoán của mô hình và giá trị thực tế trên một mẫu dữ liệu.
Binary Cross-Entropy (BCE)	Hàm mất mát tiêu chuẩn dùng cho Logistic Regression. Nó "phạt" mô hình nặng khi dự đoán sai xác suất và thưởng khi dự đoán đúng.
Hàm lồi (Convex)	Một hàm số có đồ thị hình giống chữ U, chỉ có một điểm cực tiểu duy nhất. Hàm mất mát BCE là hàm lồi, đảm bảo Gradient Descent tìm được giải pháp tốt nhất.
Hàm không lồi (Non-convex)	Một hàm số có nhiều cực tiểu địa phương. Dùng MSE cho Logistic Regression sẽ tạo ra hàm này, khiến thuật toán tối ưu có thể bị kẹt.
Gradient Descent	Thuật toán tối ưu dùng để tìm giá trị tham số làm cho hàm loss nhỏ nhất, bằng cách đi ngược hướng đạo hàm.
Batch Gradient Descent	Một phiên bản của Gradient Descent, trong đó đạo hàm được tính trung bình trên toàn bộ mẫu dữ liệu trước khi cập nhật.
Siêu tham số	Các giá trị được con người thiết lập trước khi huấn luyện, ví dụ như learning rate hay số Epoch, để kiểm soát quá trình huấn luyện.
Learning Rate	Một siêu tham số, ký hiệu là η, quyết định "bước đi" lớn hay nhỏ khi cập nhật tham số trong mỗi vòng lặp Gradient Descent.
Epoch	Một epoch có nghĩa là mô hình đã học hỏi từ toàn bộ tất cả N mẫu dữ liệu trong tập huấn luyện đúng một lần.

Bảng Ký hiệu Toán học

Ký hiệu	Ý nghĩa
N	Tổng số mẫu dữ liệu trong tập huấn luyện.
x	Biến đầu vào, hay đặc trưng của dữ liệu.
y	Biến đầu ra, hay nhãn thực tế.
(x_i, y_i)	Mẫu dữ liệu thứ i trong tập huấn luyện. x_i là đầu vào và y_i là nhãn thực tế của mẫu đó.
w	Trọng số của mô hình. Đây là tham số mô hình học được, cho biết x ảnh hưởng nhiều hay ít và tích cực hay tiêu cực đến kết quả.
b	Độ chệch của mô hình. Đây là tham số mô hình học được, giúp điều chỉnh đường biên quyết định.
z_i	Đầu vào tuyến tính, còn gọi là logit. Đây là kết quả của phép tính $z_i = wx_i + b$ trước khi đưa vào hàm Sigmoid.
$g(z)$	Hàm Sigmoid, hay hàm Logistic. Đây là hàm $g(z) = \frac{1}{1+e^{-z}}$ dùng để biến z thành xác suất.
$\hat{y}_i$	Xác suất dự đoán. Đây là đầu ra của mô hình sau khi đưa z_i qua hàm Sigmoid, $\hat{y}_i = g(z_i)$. Nó biểu thị xác suất $y_i = 1$.
$P(y = 1\|x)$	Ký hiệu toán học của xác suất có điều kiện, đọc là "xác suất $y = 1$ khi biết đầu vào là x". Đây chính là giá trị mà $\hat{y}_i$ cố gắng dự đoán.
L	Hàm mất mát Binary Cross-Entropy, đo lường lỗi của dự đoán $\hat{y}$ so với nhãn y.
$\log(\cdot)$	Hàm Logarit tự nhiên. Đây là thành phần chính trong hàm mất mát Binary Cross-Entropy, dùng để "phạt" rất nặng các dự đoán sai.
$\frac{\partial L}{\partial w}$	Đạo hàm riêng của hàm L theo trọng số w. Nó cho biết L thay đổi nhanh thế nào khi w thay đổi, và là la bàn cho Gradient Descent.
$\frac{\partial L}{\partial b}$	Đạo hàm riêng của hàm L theo b. Nó cho biết L thay đổi nhanh thế nào khi b thay đổi.
η	Hệ số học, đọc là "eta". Đây là một siêu tham số, quyết định tốc độ cập nhật w và b trong Gradient Descent.

III. Từ Linear Regression đến Logistic Regression

III.1. Ôn tập nhanh Linear Regression

Hãy cùng ôn tập lại các kiến thức về Linear Regression mà chúng ta đã tìm hiểu ở bài trước:

1. **Phương trình dự đoán:** Chúng ta dùng một hàm tuyến tính, về cơ bản là một đường thẳng, để mô hình hóa mối quan hệ và dự đoán giá trị $\hat{y}$:

$$f(x) = \hat{y} = wx + b.$$

2. **Hàm Loss:** Để đánh giá mô hình của chúng ta "tốt" hay "tệ" như thế nào, chúng ta cần một hàm lỗi $L(w, b)$. Mục tiêu là tìm w và b sao cho hàm này đạt giá trị nhỏ nhất. Trong Linear Regression, chúng ta dùng Mean Squared Error (MSE):

$$L(w, b) = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2 = \frac{1}{N}\sum_{i=1}^{N}(wx_i + b - y_i)^2.$$

   Hàm này tính trung bình của bình phương chênh lệch giữa giá trị dự đoán và giá trị thực tế trên toàn bộ N mẫu dữ liệu.

3. **Tối ưu hóa:** Sau khi có hàm loss, chúng ta sử dụng thuật toán Gradient Descent để tìm w và b tối ưu. Thuật toán này cập nhật w và b một cách lặp đi lặp lại bằng cách đi ngược hướng đạo hàm (độ dốc) của hàm loss:

$$w = w - \eta\frac{\partial L}{\partial w},$$

$$b = b - \eta\frac{\partial L}{\partial b}.$$

Vấn đề còn tồn tại: Linear Regression hoạt động tốt cho việc dự đoán các giá trị liên tục. Tuy nhiên, khi áp dụng vào bài toán phân loại, đầu ra $\hat{y} = wx + b$ của nó là một số thực bất kỳ (từ $-\infty$ đến $+\infty$). Giá trị này không thể được diễn giải trực tiếp là một xác suất (vốn dĩ phải nằm trong đoạn $[0, 1]$). Đây chính là điểm mấu chốt: chúng ta cần một cách để biến đổi giá trị $wx + b$ này vào khoảng $(0, 1)$.

III.2. Hàm Sigmoid

Như đã phân tích, chúng ta cần một hàm số nhận đầu vào là $z = wx + b$ (một số thực bất kỳ từ $-\infty$ đến $+\infty$) và "nén" nó một cách mượt mà vào khoảng $(0, 1)$ để có thể diễn giải là một xác suất.

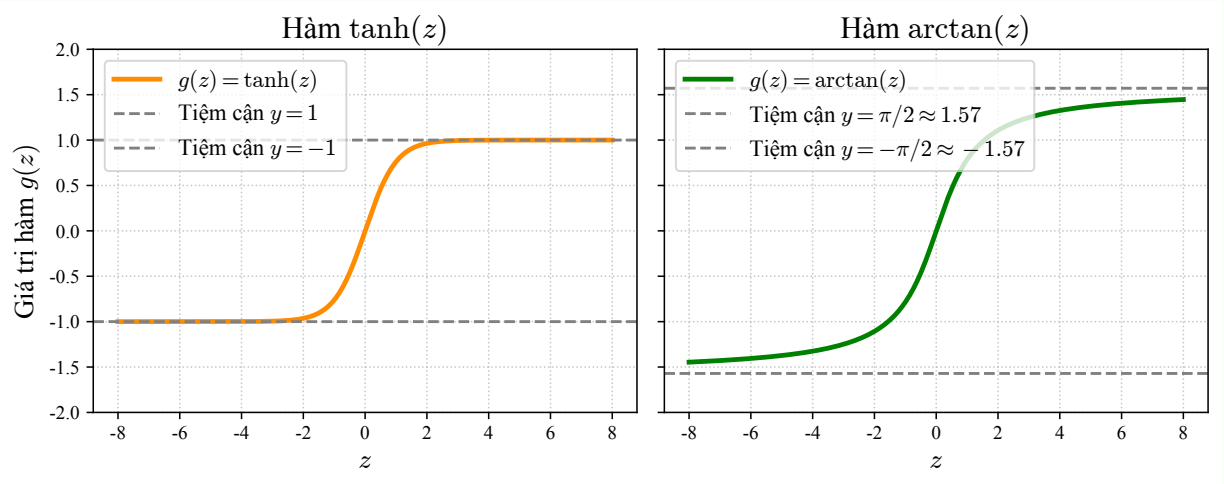
Có thể bạn chưa biết: "Sigmoid" vs. "Logistic"

Nhiều người thường nhầm lẫn giữa hai thuật ngữ này, và đây là một điểm thú vị có thể được làm rõ như sau:

- **Hàm Sigmoid:** Đây là tên gọi chung cho cả một họ các hàm số có chung đặc điểm là đồ thị hình chữ "S" (S-shaped). Chúng có đặc tính là ánh xạ một đầu vào số thực bất kỳ thành một giá trị đầu ra bị chặn trong một khoảng nhất định.
- **Hàm Logistic:** Đây là một thành viên cụ thể và cũng là thành viên nổi tiếng nhất trong họ Sigmoid. Nó có công thức chính xác là $g(z) = \frac{1}{1+e^{-z}}$ và ánh xạ đầu ra vào khoảng $(0, 1)$.

Trong học máy và đặc biệt là trong mô hình **Logistic** Regression (Hồi quy **Logistic**), hàm **Logistic** được sử dụng phổ biến đến nỗi người ta thường gọi nó là "hàm Sigmoid".
$ { }
Một số ví dụ khác về các hàm cũng thuộc họ Sigmoid bao gồm:

- **Hàm Hyperbolic Tangent (tanh):** Có công thức $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$. Nó cũng có hình chữ S nhưng có giá trị đầu ra trong khoảng $(-1, 1)$.
- **Hàm Arctangent (arctan):** Có giá trị đầu ra trong khoảng $(-\pi/2, \pi/2)$.



Tóm lại: Trong bài học này, khi chúng ta nói "Hàm Sigmoid", chúng ta đang đề cập đến "Hàm Logistic" $g(z) = \frac{1}{1+e^{-z}}$.

Khi đã làm rõ điều đó, chúng ta sẽ sử dụng hàm Logistic cho mô hình của mình, có công thức như sau:

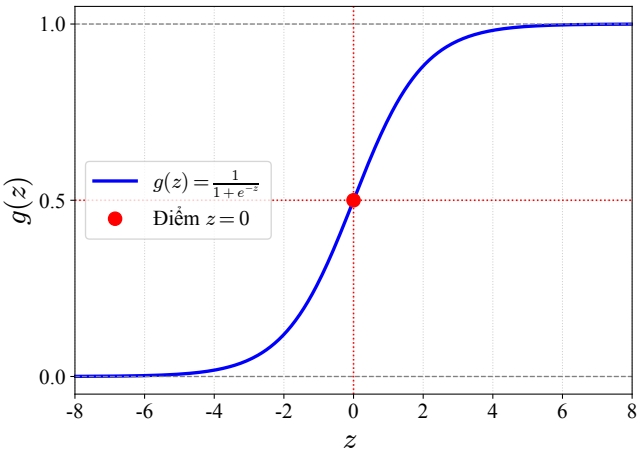
$$g(z) = \frac{1}{1 + e^{-z}}.$$

Trong đó, z chính là đầu ra của hàm tuyến tính mà chúng ta đã quen thuộc: $z = wx + b$. Khi

kết hợp lại, phương trình dự đoán $\hat{y}$ của Logistic Regression có dạng:

$$\hat{y} = g(wx + b) = \frac{1}{1 + e^{-(wx+b)}}.$$

Để hiểu tại sao hàm này lại hữu ích, chúng ta hãy cùng xem đồ thị và các tính chất toán học của nó:



Hình 3: Đồ thị của hàm $g(z) = \frac{1}{1+e^{-z}}$.

- **Khi z rất lớn ($z \to +\infty$):** Giá trị e^{-z} sẽ tiến rất nhanh về 0. Do đó, $g(z) \approx \frac{1}{1+0} = 1$.
- **Khi z rất nhỏ ($z \to -\infty$):** Giá trị e^{-z} sẽ trở nên cực kỳ lớn (tiến đến $+\infty$). Khi đó, mẫu số $(1+e^{-z})$ cũng tiến đến $+\infty$, và $g(z) \approx \frac{1}{+\infty} \approx 0$.
- **Khi $z = 0$:** Ngay tại trung tâm, ta có $e^{-0} = 1$. Do đó, $g(z) = \frac{1}{1+1} = 0.5$.

Như vậy, bất kể z là bao nhiêu, hàm Logistic luôn trả về $\hat{y}$ nằm trong khoảng $(0, 1)$. Giờ đây, chúng ta có thể diễn giải $\hat{y}$ một cách hợp lý: đó chính là xác suất dự đoán của mô hình rằng mẫu đó thuộc về lớp 1, ký hiệu là $P(y = 1|x)$.

III.2.1. Ngưỡng quyết định

Sau khi mô hình trả về một xác suất $\hat{y}$ (ví dụ: 0.7, nghĩa là 70% khả năng Đậu), làm thế nào để chúng ta đưa ra quyết định cuối cùng là 0 (Trượt) hay 1 (Đậu)?

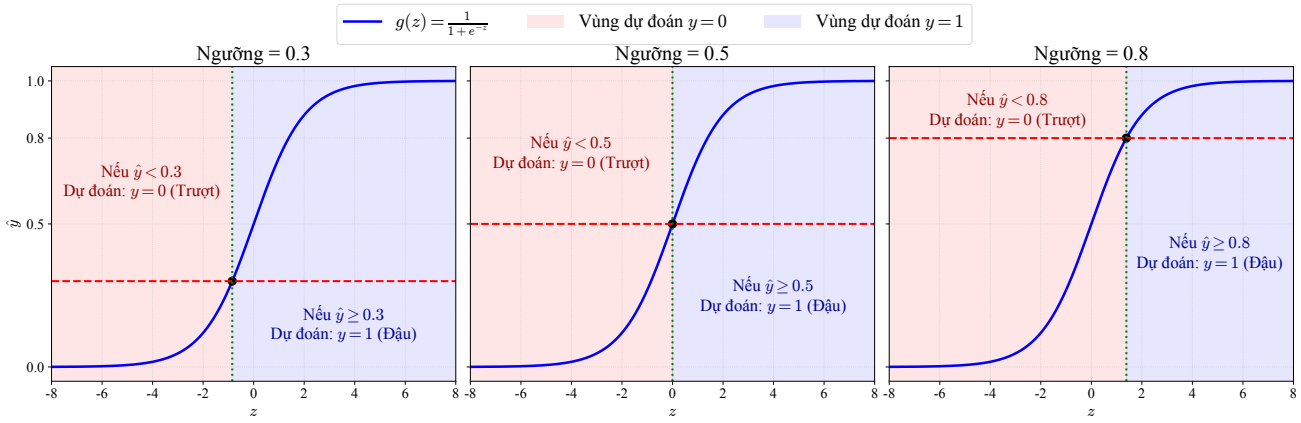
Chúng ta chỉ cần đặt ra một ngưỡng quyết định (threshold). Giá trị thông dụng nhất là 0.5:

- Nếu $\hat{y} \geq 0.5$ (xác suất Đậu từ 50% trở lên): Ta dự đoán $y = 1$ (Đậu).
- Nếu $\hat{y} < 0.5$ (xác suất Đậu dưới 50%): Ta dự đoán $y = 0$ (Trượt).

Như được minh họa rõ ràng trên Hình 4, dựa trên tính chất của hàm Logistic (Sigmoid), ngưỡng $\hat{y} = 0.5$ này tương ứng chính xác với $z = 0$. Điều này có nghĩa là:

- Nếu $z = wx + b \geq 0$, dự đoán là 1.
- Nếu $z = wx + b < 0$, dự đoán là 0.

Đường ranh giới $wx + b = 0$ này được gọi là đường biên quyết định của mô hình.



Hình 4: Minh họa các ngưỡng quyết định. Đường nét đứt màu đỏ là ngưỡng xác suất. Vùng màu xanh dương dẫn đến dự đoán $y = 1$ (Đậu), và vùng màu đỏ dẫn đến dự đoán $y = 0$ (Trượt).

III.3. Hàm mất mát mới: Binary Cross-Entropy

Ở phần trước, chúng ta đã có một hàm dự đoán $\hat{y}_i = g(z_i)$ cho ra một xác suất. Bây giờ, chúng ta cần một hàm mất mát để đo lường xác suất dự đoán "tốt" hay "tệ" như thế nào so với nhãn thực tế y_i (là 0 hoặc 1).

Một câu hỏi tự nhiên là: "Tại sao ta không dùng Mean Squared Error (MSE) như của Linear Regression?"

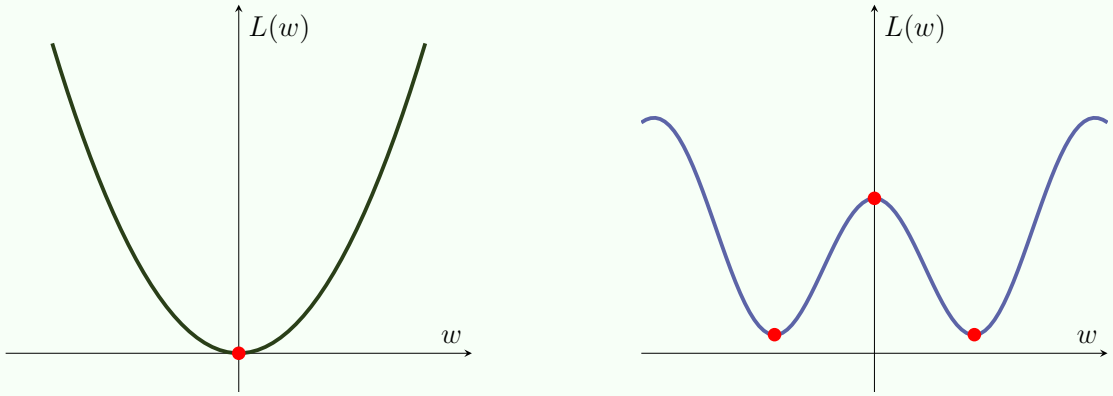
$$\text{MSE:} \quad L(w,b) = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2 = \frac{1}{N}\sum_{i=1}^{N}\left(\frac{1}{1+e^{-(wx_i+b)}} - y_i\right)^2.$$

Lý do là khi kết hợp MSE với hàm Sigmoid, hàm $L(w,b)$ sẽ không còn là một hàm lồi (non-convex) nữa. Đồ thị của nó sẽ có nhiều cực tiểu địa phương. Nếu dùng Gradient Descent, thuật toán có thể bị "kẹt" ở một thung lũng cạn mà không tìm được cực tiểu toàn cục, dẫn đến mô hình không tối ưu.

Lồi (Convex) vs. Không lồi (Non-convex)

Để hiểu tại sao điều này lại quan trọng, hãy tưởng tượng Gradient Descent là một quả bóng đang lăn xuống dốc, và hàm mất mát chính là địa hình của con dốc đó:

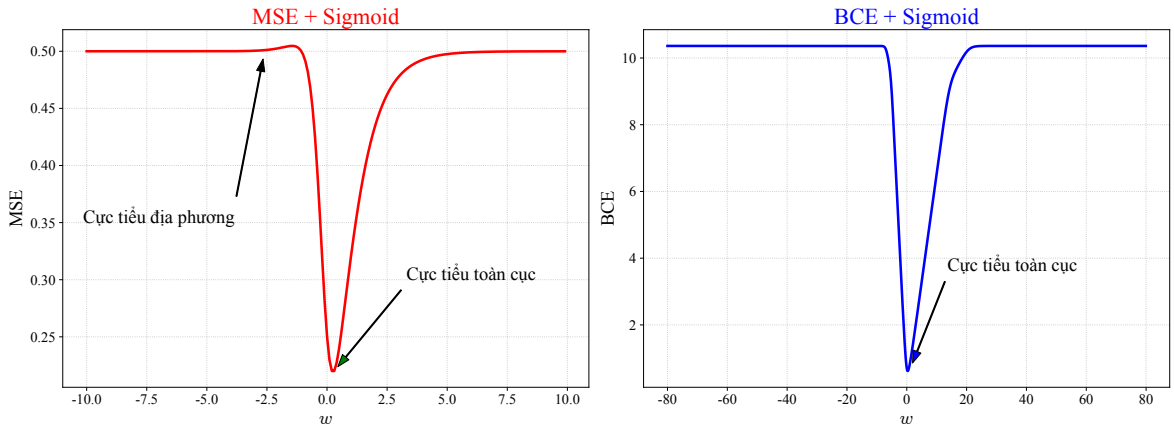
- **Hàm Lồi (Convex):** Như Hình (a) bên dưới, địa hình của hàm lồi giống như một cái bát trơn. Bất kể bạn thả quả bóng từ vị trí nào, nó sẽ luôn lăn xuống điểm thấp nhất duy nhất ở đáy bát. Điểm này chính là Cực tiểu toàn cục (global minimum).
- **Hàm Không Lồi (Non-convex):** Như Hình (b), địa hình của hàm không lồi giống như một dãy đồi núi. Nếu bạn thả quả bóng, nó có thể lăn xuống thung lũng gần nhất và bị "kẹt" ở đó (điểm Cực tiểu địa phương). Quả bóng không thể đi đến một thung lũng khác còn sâu hơn nhiều ở bên kia ngọn đồi (điểm Cực tiểu toàn cục).



(a) Hàm lồi (Convex). (b) Hàm không lồi (Non-convex).

Hình 5: So sánh tính lồi của các hàm. (a) Hàm lồi chỉ có một điểm đáy duy nhất (màu đỏ). (b) Hàm không lồi có nhiều "thung lũng", gồm cực tiểu địa phương (màu cam) và cực tiểu toàn cục (màu đỏ).

Hình 6 minh họa rõ vấn đề này.



Hình 6: So sánh tính convex của MSE và BCE khi kết hợp với hàm Sigmoid. (Trái) MSE + Sigmoid tạo ra hàm mất mát non-convex. (Phải) BCE + Sigmoid tạo ra hàm mất mát convex.

Thay vào đó, chúng ta sử dụng một hàm mất mát được thiết kế riêng cho bài toán phân loại xác suất, đó là **Binary Cross-Entropy (BCE)**, hay còn gọi là **Log Loss**.

III.3.1. Công thức cho một mẫu

Đối với một mẫu dữ liệu (x_i, y_i), hàm mất mát L_i được định nghĩa như sau:

$$L_i(\hat{y}_i, y_i) = -[y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)].$$

Công thức này có vẻ phức tạp, nhưng nó thực chất là một cách viết gộp thông minh cho hai trường hợp riêng biệt. Hãy cùng phân tích:

Trường hợp 1: Nhãn thực tế $y_i = 1$ (Sinh viên Đậu)

Khi $y_i = 1$, ta thay vào công thức L_i:

$$L_i = -[1 \cdot \log(\hat{y}_i) + (1-1) \cdot \log(1-\hat{y}_i)],$$

$$L_i = -[1 \cdot \log(\hat{y}_i) + 0 \cdot \log(1-\hat{y}_i)] = -\log(\hat{y}_i).$$

Vậy khi nhãn là 1, chúng ta chỉ cần quan tâm đến $L_i = -\log(\hat{y}_i)$.

- Nếu mô hình dự đoán $\hat{y}_i \to 1$ (rất gần 1, dự đoán đúng): Mất mát $L_i = -\log(1) \approx 0$. Đây là mất mát rất nhỏ, vì mô hình đã làm tốt.
- Nếu mô hình dự đoán $\hat{y}_i \to 0$ (rất gần 0, dự đoán sai): Mất mát $L_i = -\log(0) \to +\infty$. Đây là mất mát vô cùng lớn, là một sự "phạt" nặng cho mô hình vì đã dự đoán sai hoàn toàn.

Trường hợp 2: Nhãn thực tế $y_i = 0$ (Sinh viên Trượt)

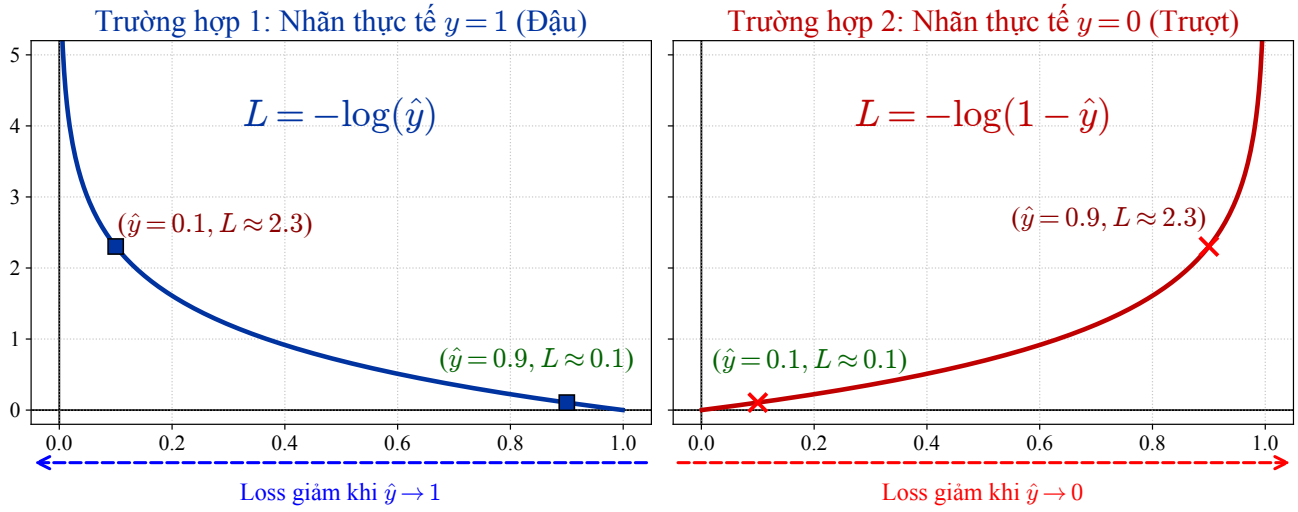
Khi $y_i = 0$, ta thay vào công thức L_i:

$$L_i = -[0 \cdot \log(\hat{y}_i) + (1-0) \cdot \log(1-\hat{y}_i)],$$

$$L_i = -[0 + 1 \cdot \log(1-\hat{y}_i)] = -\log(1-\hat{y}_i).$$

Vậy khi nhãn là 0, chúng ta chỉ cần quan tâm đến $L_i = -\log(1-\hat{y}_i)$.

- Nếu mô hình dự đoán $\hat{y}_i \to 0$ (rất gần 0, dự đoán đúng): Mất mát $L_i = -\log(1-0) \approx -\log(1) \approx 0$. Mất mát rất nhỏ, mô hình làm tốt.
- Nếu mô hình dự đoán $\hat{y}_i \to 1$ (rất gần 1, dự đoán sai): Mất mát $L_i = -\log(1-1) \to -\log(0) \to +\infty$. Mất mát vô cùng lớn, "phạt" nặng mô hình.



Hình 7: Trực quan hóa hàm mất mát Binary Cross-Entropy. Khi nhãn thực tế $y = 1$, ta sử dụng hàm mất mát $L = -\log(\hat{y})$. Khi nhãn thực tế $y = 0$, ta sử dụng hàm mất mát $L = -\log(1-\hat{y})$.

Toàn bộ phân tích ở trên được minh họa rõ ràng tại Hình 7, cho thấy hàm Binary Cross-Entropy hoạt động chính xác như chúng ta mong muốn.

III.3.2. Hàm tổng quát

Hàm mất mát $L(w, b)$ tổng quát cho toàn bộ N mẫu dữ liệu đơn giản là giá trị trung bình của tất cả các mất mát L_i trên từng mẫu:

$$L(w, b) = \frac{1}{N} \sum_{i=1}^{N} L_i = -\frac{1}{N} \sum_{i=1}^{N} (y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)).$$

Không giống hàm MSE vốn trở nên non-convex khi kết hợp với Sigmoid, hàm mất mát $L(w, b)$ sử dụng BCE này được chứng minh là một hàm convex.

Nhờ đặc điểm này, chúng ta đã giải quyết được vấn đề "cực tiểu địa phương" đã nêu ở đầu. Giờ đây, khi áp dụng thuật toán Gradient Descent, chúng ta được đảm bảo rằng nó chắc chắn sẽ tìm được một và chỉ một điểm cực tiểu toàn cục, giúp mô hình đạt được hiệu năng tối ưu nhất.

III.4. Gradient Descent cho Logistic Regression

Sau khi đã có hàm mất mát $L(w, b)$, mục tiêu của chúng ta là tìm w và b sao cho L nhỏ nhất. Chúng ta lại sử dụng thuật toán quen thuộc: **Gradient Descent**.

Để áp dụng Gradient Descent, chúng ta cần tính đạo hàm riêng của L theo w và b. Quá trình tính toán đạo hàm này sử dụng quy tắc chuỗi vì các hàm được lồng vào nhau:

- Hàm $L(w, b)$ phụ thuộc vào $\hat{y}_i$.
- Xác suất $\hat{y}_i$ phụ thuộc vào z_i.
- Giá trị z_i phụ thuộc vào w và b.

III.4.1. Chi tiết phép toán đạo hàm

Ta có các công thức:

$$L = -\frac{1}{N} \sum_{i=1}^{N} [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)],$$

$$\hat{y}_i = g(z_i) = \frac{1}{1 + e^{-z_i}},$$

$$z_i = wx_i + b.$$

Vì hàm L là trung bình của các L_i, chúng ta có thể tập trung tính đạo hàm cho một mẫu L_i trước, sau đó lấy trung bình $1/N$ sau.

Mục tiêu 1: tìm $\dfrac{\partial L}{\partial w}$

Chúng ta sẽ áp dụng quy tắc chuỗi:

$$\frac{\partial L_i}{\partial w} = \frac{\partial L_i}{\partial \hat{y}_i} \cdot \frac{\partial \hat{y}_i}{\partial z_i} \cdot \frac{\partial z_i}{\partial w}.$$

Ta sẽ chia nhỏ và giải quyết từng thành phần:

1. **Phần 1: $\frac{\partial L_i}{\partial \hat{y}_i}$ (Đạo hàm của Loss theo dự đoán)**

   $$L_i = -[y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)].$$

   Lấy đạo hàm L_i theo $\hat{y}_i$:

   $$\frac{\partial L_i}{\partial \hat{y}_i} = -\left[y_i \cdot \frac{1}{\hat{y}_i} + (1 - y_i) \cdot \frac{1}{1 - \hat{y}_i} \cdot (-1)\right] = -\left[\frac{y_i}{\hat{y}_i} - \frac{1 - y_i}{1 - \hat{y}_i}\right].$$

   Quy đồng mẫu số và rút gọn:

   $$\frac{\partial L_i}{\partial \hat{y}_i} = \frac{-(y_i(1 - \hat{y}_i) - \hat{y}_i(1 - y_i))}{\hat{y}_i(1 - \hat{y}_i)} = \frac{-(y_i - y_i\hat{y}_i - \hat{y}_i + y_i\hat{y}_i)}{\hat{y}_i(1 - \hat{y}_i)}.$$

   $$\frac{\partial L_i}{\partial \hat{y}_i} = \frac{-(y_i - \hat{y}_i)}{\hat{y}_i(1 - \hat{y}_i)} = \frac{\hat{y}_i - y_i}{\hat{y}_i(1 - \hat{y}_i)}.$$

2. **Phần 2: $\frac{\partial \hat{y}_i}{\partial z_i}$ (Đạo hàm của hàm Sigmoid)**

   Đây là một thuộc tính toán học rất đẹp của hàm Sigmoid $g(z)$:

   $$\hat{y}_i = g(z_i) = (1 + e^{-z_i})^{-1},$$

   $$\frac{\partial \hat{y}_i}{\partial z_i} = -1 \cdot (1 + e^{-z_i})^{-2} \cdot (e^{-z_i} \cdot -1) = \frac{e^{-z_i}}{(1 + e^{-z_i})^2}.$$

   Chúng ta có thể biến đổi nó một cách thông minh:

   $$= \left(\frac{1}{1 + e^{-z_i}}\right) \cdot \left(\frac{e^{-z_i}}{1 + e^{-z_i}}\right) = \left(\frac{1}{1 + e^{-z_i}}\right) \cdot \left(\frac{1 + e^{-z_i} - 1}{1 + e^{-z_i}}\right),$$

   $$= \left(\frac{1}{1 + e^{-z_i}}\right) \cdot \left(1 - \frac{1}{1 + e^{-z_i}}\right).$$

   Vì $\hat{y}_i = g(z_i) = \frac{1}{1+e^{-z_i}}$, nên ta có kết quả:

   $$\frac{\partial \hat{y}_i}{\partial z_i} = g(z_i) \cdot (1 - g(z_i)) = \hat{y}_i(1 - \hat{y}_i).$$

3. **Phần 3: $\frac{\partial z_i}{\partial w}$ (Đạo hàm của phần tuyến tính)**

   $$z_i = wx_i + b,$$

   $$\frac{\partial z_i}{\partial w} = x_i.$$

Kết hợp lại: Bây giờ, chúng ta nhân cả 3 phần lại với nhau:

$$\frac{\partial L_i}{\partial w} = \left(\frac{\partial L_i}{\partial \hat{y}_i}\right) \cdot \left(\frac{\partial \hat{y}_i}{\partial z_i}\right) \cdot \left(\frac{\partial z_i}{\partial w}\right),$$

$$\frac{\partial L_i}{\partial w} = \left(\frac{\hat{y}_i - y_i}{\hat{y}_i(1-\hat{y}_i)}\right) \cdot (\hat{y}_i(1-\hat{y}_i)) \cdot (x_i).$$

Điều kỳ diệu xảy ra ở đây: các thành phần mẫu số và tử số $\hat{y}_i(1-\hat{y}_i)$ triệt tiêu lẫn nhau!

$$\frac{\partial L_i}{\partial w} = (\hat{y}_i - y_i)x_i.$$

Đây mới chỉ là đạo hàm cho một mẫu L_i. Để có đạo hàm cho hàm L, ta chỉ cần lấy trung bình trên tất cả N mẫu:

$$\frac{\partial L}{\partial w} = \frac{1}{N}\sum_{i=1}^{N}\frac{\partial L_i}{\partial w} = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)x_i.$$

Mục tiêu 2: Tìm $\frac{\partial L}{\partial b}$

Quá trình này hoàn toàn tương tự, chỉ khác ở Phần 3:

$$\frac{\partial z_i}{\partial b} = 1.$$

Khi kết hợp lại:

$$\frac{\partial L_i}{\partial b} = \left(\frac{\hat{y}_i - y_i}{\hat{y}_i(1-\hat{y}_i)}\right) \cdot (\hat{y}_i(1-\hat{y}_i)) \cdot (1) = (\hat{y}_i - y_i).$$

Lấy trung bình trên N mẫu, ta có:

$$\frac{\partial L}{\partial b} = \frac{1}{N}\sum_{i=1}^{N}\frac{\partial L_i}{\partial b} = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i).$$

III.4.2. Kết luận

Sau khi rút gọn tất cả các phép toán phức tạp, kết quả cuối cùng lại khá đơn giản:

$$\frac{\partial L}{\partial w} = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)x_i,$$

$$\frac{\partial L}{\partial b} = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i).$$

Nếu bạn thấy quen thuộc thì đó là vì các công thức đạo hàm này trông giống hệt như công thức đạo hàm của Linear Regression! Đây là một sự trùng hợp rất thú vị và tiện lợi.

So sánh Gradient Descent: Linear vs. Logistic

Cả hai mô hình đều có chung một công thức cập nhật gradient về mặt hình thức:

$$\text{gradient} \sim (\text{Dự đoán} - \text{Thực tế}) \times \text{Đầu vào}$$

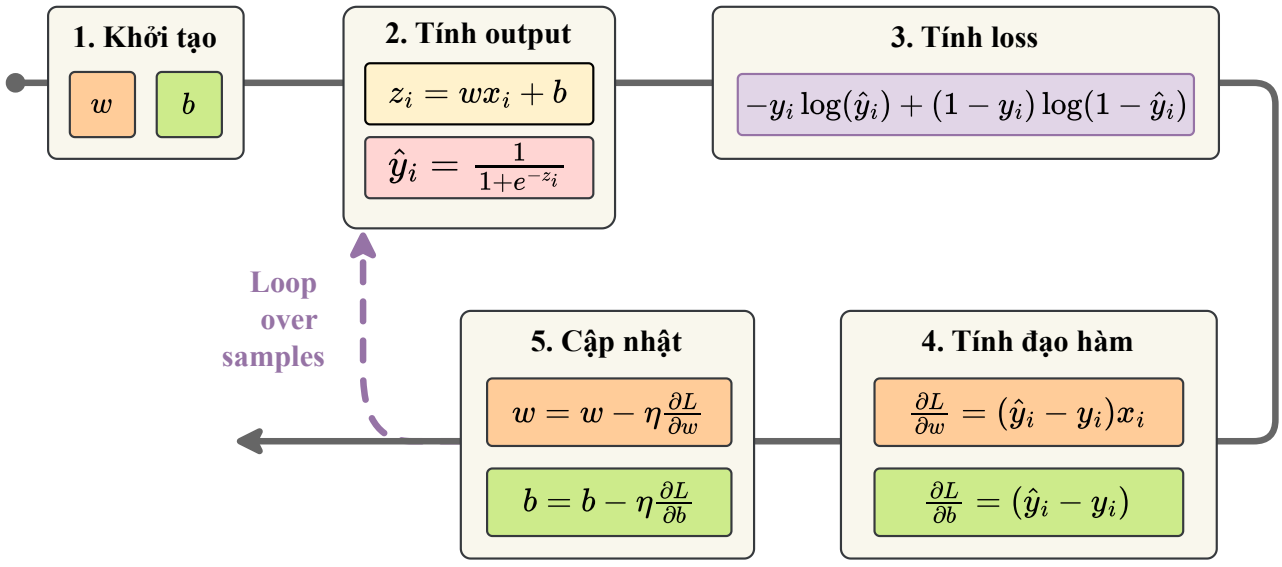
Tuy nhiên, có một điểm khác biệt cốt lõi nằm ở định nghĩa của dự đoán ($\hat{y}_i$):

- **Trong Linear Regression:**
$$\hat{y}_i = wx_i + b.$$
Giá trị $\hat{y}_i$ là một số thực bất kỳ (ví dụ: $\hat{y}_i = 105.5$).
- **Trong Logistic Regression:**
$$\hat{y}_i = g(wx_i + b) = \frac{1}{1 + e^{-(wx_i+b)}}.$$
Giá trị $\hat{y}_i$ là một xác suất luôn nằm trong khoảng $(0, 1)$ (ví dụ: $\hat{y}_i = 0.95$).

Mặc dù công thức đạo hàm trông giống nhau, nhưng giá trị của $(\hat{y}_i - y_i)$ được tính hoàn toàn khác nhau. Trong Linear, đó là $(wx_i + b) - y_i$. Trong Logistic, đó là $g(wx_i + b) - y_i$. Sự khác biệt trong cách tính $\hat{y}_i$ này dẫn đến quá trình hội tụ hoàn toàn khác nhau.

III.5. Quy trình huấn luyện

Quá trình huấn luyện cho Logistic Regression về cơ bản khá giống với Linear Regression, khi cũng sử dụng Batch Gradient Descent để tối ưu các tham số w và b. Toàn bộ các bước được minh họa trong Hình 8.



Hình 8: Quy trình tổng quát huấn luyện mô hình Logistic Regression.

1. **Khởi tạo tham số**
   - Khởi tạo giá trị ban đầu cho w và b (ví dụ: $w = 0$, $b = 0$).
   - Chọn các siêu tham số:
     - Hệ số học η (ví dụ: 0.01).
     - Số vòng lặp (epochs), tức là số lần mô hình sẽ học qua toàn bộ N mẫu dữ liệu.
2. **Lặp lại quá trình huấn luyện**

   Lặp theo số *epoch* đã chọn:
   (a) Tính dự đoán $\hat{y}_i$ cho **tất cả** N mẫu:
   - Tính đầu vào tuyến tính: $z_i = wx_i + b$.
   - Tính xác suất dự đoán: $\hat{y}_i = g(z_i) = \frac{1}{1+e^{-z_i}}$.

   (b) Tính hàm mất mát L trên toàn bộ tập dữ liệu:

$$L = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)].$$

   (c) Tính đạo hàm trung bình trên toàn bộ N mẫu (Gradient):

$$\frac{\partial L}{\partial w} = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)x_i, \qquad \frac{\partial L}{\partial b} = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i).$$

   (d) Cập nhật w và b đồng thời (Gradient Descent):

$$w = w - \eta \cdot \frac{\partial L}{\partial w}, \qquad b = b - \eta \cdot \frac{\partial L}{\partial b.}$$

3. **Kết thúc**

   Sau khi hoàn thành, chúng ta thu được các giá trị w và b đã được tối ưu.

Như vậy, bằng cách lặp quy trình trên, mô hình sẽ dần "học" được các tham số tối ưu. Sau mỗi epoch, giá trị của hàm mất mát $L(w, b)$ sẽ giảm xuống, và đường biên quyết định $wx + b = 0$ sẽ ngày càng di chuyển về vị trí tốt nhất để phân chia hai lớp dữ liệu 0 và 1.

Bây giờ, hãy cùng áp dụng toàn bộ lý thuyết này vào một ví dụ thực hành cụ thể dưới đây!

IV. Thực hành

IV.1. Phát biểu bài toán

Xét bài toán ví dụ: dự đoán sinh viên thi Đậu (1) hay Trượt (0) dựa trên số giờ ôn tập. Chúng ta sẽ lấy ra 6 điểm dữ liệu ($N = 6$) để thực hành tính toán:

Sinh viên (i)	Số giờ học (x_i)	Kết quả (y_i)
1	1.0	0
2	2.5	0
3	4.0	0
4	6.0	1
5	7.0	1
6	8.0	1

Bảng 1: Bộ dữ liệu mẫu (N=6) về kết quả thi.

Mục tiêu: Chúng ta sẽ thực hiện tính toán một vòng lặp (epoch) của Batch Gradient Descent để tìm ra các giá trị w và b mới, bắt đầu từ giá trị khởi tạo.

IV.2. Chi tiết tính toán

Chúng ta sẽ tuân thủ quy trình huấn luyện đã được tóm tắt ở Mục III.5..

1. **Bước 1: Khởi tạo tham số**

   Để đơn giản, ta khởi tạo các giá trị ban đầu:

   - Trọng số $w = 0$,
   - Độ chệch $b = 0$.

   Đồng thời, ta chọn siêu tham số:

   - Hệ số học $\eta = 0.2$.

2. **Bước 2: Lặp lại quá trình huấn luyện (Thực hiện Epoch 1)**

   Chúng ta thực hiện các bước tính toán cho vòng lặp đầu tiên.

   (a) **Bước 2a: Tính dự đoán $\hat{y}_i$ cho tất cả $N = 6$ mẫu**

   Đầu tiên, ta tính giá trị tuyến tính $z_i = wx_i + b$. Vì $w = 0$ và $b = 0$, nên $z_i = (0 \cdot x_i) + 0 = 0$ cho tất cả các mẫu $i = 1, ..., 6$.

   Tiếp theo, ta tính xác suất dự đoán $\hat{y}_i$ bằng hàm Sigmoid:

$$\hat{y}_i = g(z_i) = g(0) = \frac{1}{1 + e^{-0}} = \frac{1}{1 + 1} = 0.5.$$

Diễn giải: Tất cả 6 sinh viên đều có dự đoán xác suất Đậu là 0.5 (tức 50%). Điều này hợp lý vì mô hình được khởi tạo $w = 0, b = 0$ nên nó hoàn toàn không biết gì và đưa ra dự đoán "50/50" cho mọi trường hợp.

(b) **Bước 2b: Tính hàm mất mát L**

Ta sử dụng công thức Binary Cross-Entropy tổng quát:

$$L = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)].$$

Vì ở bước 2a, ta có $\hat{y}_i = 0.5$ cho tất cả các mẫu, ta thay vào:

$$L = -\frac{1}{6}\sum_{i=1}^{6}[y_i \log(0.5) + (1 - y_i)\log(0.5)]$$

Ta nhóm $\log(0.5)$ lại:

$$L = -\frac{1}{6}\sum_{i=1}^{6}[(y_i + 1 - y_i)\log(0.5)] = -\frac{1}{6}\sum_{i=1}^{6}[1 \cdot \log(0.5)],$$

$$L = -\frac{1}{6}[6 \cdot \log(0.5)] = -\log(0.5) = \mathbf{0.6931}.$$

(c) **Bước 2c: Tính đạo hàm trung bình**

Bây giờ, chúng ta cần tính $\frac{\partial L}{\partial w}$ và $\frac{\partial L}{\partial b}$. Trước tiên chúng ta cần tính các thành phần sai số $(\hat{y}_i - y_i)$ và $(\hat{y}_i - y_i)x_i$ cho từng mẫu.

Để cho trực quan, chúng ta có thể tổng hợp các giá trị này trong một bảng:

Mẫu (i)	x_i	y_i	$\hat{y}_i$	**Sai số** $(\hat{y}_i - y_i)$	**Thành phần** w $(\hat{y}_i - y_i)x_i$
1	1.0	0	0.5	$0.5 - 0 = \mathbf{0.5}$	$0.5 \times 1.0 = \mathbf{0.5}$
2	2.5	0	0.5	$0.5 - 0 = \mathbf{0.5}$	$0.5 \times 2.5 = \mathbf{1.25}$
3	4.0	0	0.5	$0.5 - 0 = \mathbf{0.5}$	$0.5 \times 4.0 = \mathbf{2.0}$
4	6.0	1	0.5	$0.5 - 1 = \mathbf{-0.5}$	$-0.5 \times 6.0 = \mathbf{-3.0}$
5	7.0	1	0.5	$0.5 - 1 = \mathbf{-0.5}$	$-0.5 \times 7.0 = \mathbf{-3.5}$
6	8.0	1	0.5	$0.5 - 1 = \mathbf{-0.5}$	$-0.5 \times 8.0 = \mathbf{-4.0}$
Tổng $\sum$	-	-	-	**0.0**	$\mathbf{-6.75}$

Bảng 2: Tính toán các thành phần đạo hàm cho Epoch 1.

Dựa vào cột tổng ($\sum$) của Bảng 2, ta có:

- $\sum_{i=1}^{N}(\hat{y}_i - y_i) = 0.0$,
- $\sum_{i=1}^{N}(\hat{y}_i - y_i)x_i = (0.5 + 1.25 + 2.0) - (3.0 + 3.5 + 4.0) = 3.75 - 10.5 = \mathbf{-6.75}$.

Giờ ta có thể dễ dàng tính các đạo hàm trung bình (với $N = 6$):

$$\frac{\partial L}{\partial b} = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i) = \frac{1}{6}(0.0) = \mathbf{0},$$

$$\frac{\partial L}{\partial w} = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)x_i = \frac{1}{6}(-6.75) = \mathbf{-1.125}.$$

(d) **Bước 2d: Cập nhật w và b**
Chúng ta sử dụng công thức cập nhật Gradient Descent với $\eta = 0.2$:

$$w_{\text{mới}} = w - \eta \cdot \frac{\partial L}{\partial w} = 0 - (0.2) \cdot (-1.125) = \mathbf{0.225},$$

$$b_{\text{mới}} = b - \eta \cdot \frac{\partial L}{\partial b} = 0 - (0.2) \cdot (0) = \mathbf{0}.$$

3. **Bước 3: Kết thúc Epoch 1**
Sau khi hoàn thành, các tham số mới của mô hình là:

- $w = 0.225$,
- $b = 0$.

IV.3. Phân tích kết quả epoch 1

Hãy cùng xem mô hình đã "học" được gì chỉ sau một bước cập nhật:

- Giá trị w đã thay đổi từ 0 lên **0.225**. Đạo hàm $\frac{\partial L}{\partial w}$ là $\mathbf{-1.125}$ (âm), có nghĩa là hàm mất mát đang "nói" chúng ta rằng: "Việc *tăng* w sẽ làm *giảm* mất mát". Thuật toán Gradient Descent đã thực hiện đúng điều đó, tức là đi ngược dấu đạo hàm.
- Giá trị w mới là số dương, cho thấy mô hình đang bắt đầu "học" được rằng x (số giờ học) càng *tăng* thì xác suất Đậu cũng càng *tăng*.

Dự đoán lại với tham số mới:
Dùng $w \approx 0.225$ và $b = 0$ để dự đoán cho sinh viên 4 (người học $x_4 = 6.0$ giờ và có nhãn $y_4 = 1$):

1. Tính $z_4^{\text{mới}}$:
$$z_4^{\text{mới}} = w_{\text{mới}} \cdot x_4 + b_{\text{mới}} \approx (0.225 \cdot 6.0) + 0 = \mathbf{1.35}.$$
2. Tính $\hat{y}_4^{\text{mới}}$:
$$\hat{y}_4^{\text{mới}} = g(z_4^{\text{mới}}) = g(1.35) = \frac{1}{1 + e^{-1.35}} \approx \frac{1}{1 + 0.2592} \approx \mathbf{0.7941}.$$

Dự đoán mới là **0.7941** (tức 79.41%), đã nhích lên đáng kể so với dự đoán 0.5 ban đầu, và tiến gần hơn đến nhãn thực tế là $y_4 = 1$.

Mô hình rõ ràng đang đi đúng hướng! Nếu chúng ta lặp lại quá trình này, w và b sẽ tiếp tục được điều chỉnh, cho đến khi mô hình tìm được đường cong phù hợp để phân chia dữ liệu.

IV.4. Phân tích kết quả huấn luyện qua các epoch

Nếu chúng ta lặp lại quá trình này hàng nghìn lần, w và b sẽ tiếp tục được điều chỉnh. Bảng 3 tóm tắt quá trình học của mô hình qua 10000 epochs, với dự đoán cho mẫu $x = 6.0$.

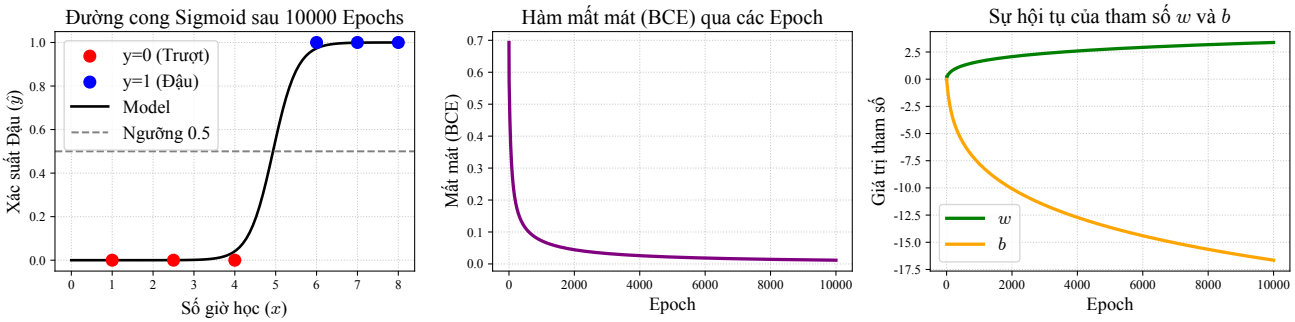
Epoch	Trọng số (w)	Độ chệch (b)	Mất mát (BCE)	Dự đoán $\hat{y}$ ($x = 6.0$)
0	0.0000	0.0000	0.6931	0.5000
1	0.2250	0.0000	0.6064	0.7941
2	0.1776	-0.0462	0.5916	0.7349
5	0.1956	-0.1540	0.5718	0.7349
10	0.2260	-0.3264	0.5413	0.7368
50	0.4280	-1.4464	0.3783	0.7542
500	1.2504	-5.8236	0.1130	0.8428
2000	2.0740	-10.0757	0.0444	0.9144
10000	3.3737	-16.6475	0.0117	0.9733

Bảng 3: Tóm tắt quá trình huấn luyện qua 10000 epochs.

Phân tích (từ Bảng 3):

- **Trọng số w:** Liên tục tăng từ 0 lên $\approx$ **3.3737**. Mô hình học rất rõ ràng rằng w dương (số giờ học tăng thì khả năng đậu tăng).
- **Độ chệch b:** Giảm dần từ 0 xuống $\approx$ **-16.6475**, được điều chỉnh để tìm ra đường ranh giới quyết định phù hợp nhất với dữ liệu.
- **Mất mát (BCE):** Giảm từ 0.6931 xuống còn **0.0117**. Đây là bằng chứng rõ ràng nhất cho thấy mô hình đang học tốt và hội tụ.
- **Dự đoán $\hat{y}$ ($x = 6.0$):** Dự đoán cho sinh viên học 6.0 giờ tăng từ 50% lên 79.4% (sau 1 epoch), và hội tụ về $\approx$ **0.9733** (97.3%), ngày càng tự tin hơn về nhãn thực tế $y = 1$.

Để trực quan hóa toàn bộ quá trình, Hình 9 biểu diễn các kết quả huấn luyện một cách rõ ràng:



Hình 9: Trực quan hóa quá trình huấn luyện Logistic Regression sau 10000 epochs.

Phân tích Hình 9:

- **(Biểu đồ giữa) Hàm mất mát (BCE):** Đường cong màu tím cho thấy hàm mất mát giảm cực kỳ nhanh trong khoảng 1000-2000 epochs đầu tiên và sau đó hội tụ từ từ về gần 0. Điều này *hoàn toàn khớp* với cột "Mất mát (BCE)" trong Bảng 3, xác nhận mô hình đang học hiệu quả.
- **(Biểu đồ phải) Sự hội tụ của tham số:** Biểu đồ này trực quan hóa hai cột đầu tiên của bảng. Ta thấy w (màu xanh lá) tăng dần từ 0 lên giá trị dương và b (màu cam) giảm dần từ 0 xuống giá trị âm. Quá trình thay đổi mạnh nhất cũng xảy ra ở các epoch đầu, sau đó tốc độ thay đổi chậm lại khi tiến gần đến giá trị hội tụ.
- **(Biểu đồ trái) Đường cong Sigmoid cuối cùng:** Đây là kết quả sau 10000 epochs, khi $w \approx 3.37$ và $b \approx -16.65$. Đường cong Sigmoid (màu đen) đã "uốn" một cách hoàn hảo để phân tách hai nhóm dữ liệu: dự đoán $\hat{y} \approx 0$ cho các điểm $y = 0$ (Trượt, màu đỏ) và $\hat{y} \approx 1$ cho các điểm $y = 1$ (Đậu, màu xanh).

V. Câu hỏi trắc nghiệm

1. Vấn đề chính khi áp dụng trực tiếp Linear Regression ($f(x) = wx + b$) cho bài toán phân loại nhị phân là gì?

   (a) Nó không thể xử lý nhãn $y = 0$.

   (b) Hàm mất mát MSE không thể tính đạo hàm.

   (c) Đầu ra $\hat{y}$ có thể nằm ngoài khoảng $[0, 1]$, khiến việc diễn giải xác suất trở nên vô nghĩa.

   (d) Nó yêu cầu w và b phải là số dương.

2. Lý do quan trọng nhất để không sử dụng hàm mất mát Mean Squared Error (MSE) cho Logistic Regression là gì?

   (a) MSE không thể tính đạo hàm với hàm Sigmoid.

   (b) MSE luôn cho giá trị mất mát là số âm.

   (c) MSE tạo ra hàm mất mát không lồi (non-convex) khi kết hợp với Sigmoid, khiến Gradient Descent có thể bị kẹt ở cực tiểu địa phương.

   (d) MSE không thể xử lý các nhãn $y = 0$ và $y = 1$ một cách chính xác.

3. Trong hàm mất mát Binary Cross-Entropy, khi nhãn thực tế $y_i = 1$, thành phần $L_i = -\log(\hat{y}_i)$ hoạt động như thế nào?

   (a) Nó phạt mô hình khi $\hat{y}_i \to 1$ và thưởng khi $\hat{y}_i \to 0$.

   (b) Nó phạt mô hình rất nặng khi dự đoán $\hat{y}_i \to 0$ (sai) và phạt nhẹ (loss ≈ 0) khi $\hat{y}_i \to 1$ (đúng).

   (c) Nó đảm bảo w và b luôn dương.

   (d) Nó chỉ đơn giản là tính sai số $(\hat{y}_i - 1)$.

4. Trong Logistic Regression, ngưỡng quyết định mặc định là $\hat{y} = 0.5$. Điều này tương ứng với giá trị nào của đầu vào tuyến tính $z = wx + b$?

   (a) $z = 1$.

   (b) $z = 0$.

   (c) $z \to +\infty$.

   (d) z phụ thuộc vào giá trị của x.

5. Một trong những kết quả "kỳ diệu" khi tính đạo hàm của Binary Cross-Entropy qua hàm Sigmoid là các thành phần $\hat{y}_i(1 - \hat{y}_i)$ bị triệt tiêu. Kết quả đạo hàm cuối cùng cho $\frac{\partial L_i}{\partial w}$ (cho một mẫu i) là gì?

   (a) $(\hat{y}_i - y_i)$.

   (b) $(\hat{y}_i - y_i)x_i$.

   (c) $\frac{\hat{y}_i - y_i}{\hat{y}_i(1-\hat{y}_i)}$.

(d) $g(z_i) \cdot (1 - y_i)$.

6. Điểm khác biệt giữa công thức đạo hàm của Logistic Regression và Linear Regression **cốt lõi** nằm ở đâu?

   (a) Logistic Regression dùng y_i là (0, 1) trong khi Linear dùng số thực.

   (b) Cách tính $\hat{y}_i$. Trong Linear, $\hat{y}_i = wx_i + b$, nhưng trong Logistic, $\hat{y}_i = g(wx_i + b)$.

   (c) Logistic Regression phải dùng η (learning rate) nhỏ hơn.

   (d) Không có khác biệt nào, chúng là một.

7. Trong ví dụ thực hành (Bảng 3), sau 10000 epochs, mô hình hội tụ với $w \approx 3.3737$ và $b \approx -16.6475$. Ý nghĩa của w dương (3.3737) là gì?

   (a) Mô hình đã học được rằng w càng lớn thì loss càng giảm.

   (b) Mô hình đã học được mối quan hệ: số giờ học càng *tăng* thì xác suất đậu càng *tăng*.

   (c) w dương là do η (learning rate) được chọn là số dương.

   (d) w dương để bù lại b bị âm, nhằm giữ cho z luôn ≈ 0.

8. Dựa trên bộ dữ liệu trong Bảng 1, giả sử ta thay đổi x_1 từ 1.0 thành 2.0 (các điểm khác giữ nguyên) và bắt đầu huấn luyện với $w = 0, b = 0$. Giá trị đạo hàm $\frac{\partial L}{\partial w}$ (trung bình trên $N = 6$ mẫu) trong Epoch 1 sẽ là bao nhiêu?

   (a) -1.125.

   (b) -1.0417.

   (c) -1.2083.

   (d) -6.25.

9. Sử dụng các tham số đã cập nhật sau Epoch 1 trong bài thực hành ($w = 0.225, b = 0$), dự đoán $\hat{y}$ cho một sinh viên mới học $x = 3.0$ giờ là bao nhiêu? (Biết $e^{-0.675} \approx 0.509$).

   (a) 0.5000.

   (b) 0.7941.

   (c) 0.6627.

   (d) 0.5560.

10. Một mô hình dự đoán xác suất đậu cho một sinh viên là $\hat{y} = 0.2$. Tuy nhiên, nhãn thực tế của sinh viên đó là $y = 1$ (Đậu). Hàm mất mát Binary Cross-Entropy L_i cho mẫu này là bao nhiêu? (Biết $\log_e(0.2) \approx -1.609$ và $\log_e(0.8) \approx -0.223$).

    (a) 1.609.

    (b) -1.609.

    (c) 0.223.

    (d) -0.223.

VI. Tài liệu tham khảo

[1] V. H. Tip. "Lesson 10: Logistic regression". [Online]. Available: https://machinelearningcoban.com/2017/01/27/logisticregression.

[2] D. R. Cox, "The regression analysis of binary sequences", *Journal of the Royal Statistical Society. Series B (Methodological)*, pp. 215–242, 1958.

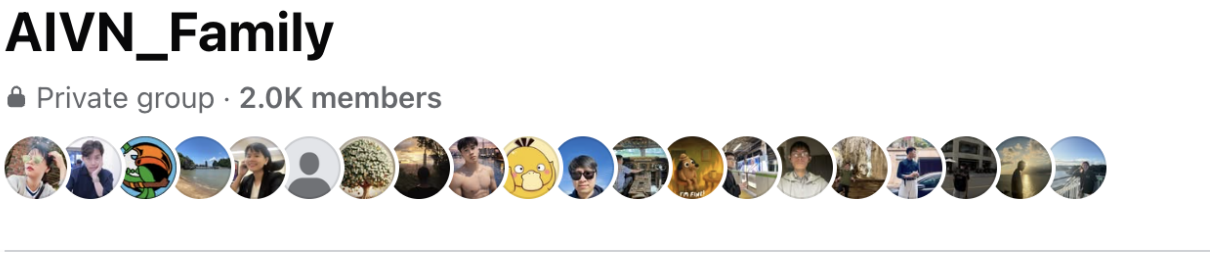
[3] J. S. Cramer, "The origins of logistic regression", University of Amsterdam, Tech. Rep., 2002, Tinbergen Institute Discussion Paper, TI 2002-119/4.

[4] Y. S. Abu-Mostafa et al., *Learning from data*. New York, NY, USA: AMLBook, 2012, vol. 4.

[5] A. Ng, *Cs229 lecture notes: Part ii: Classification and logistic regression*, Course notes, Stanford University, 2008.

Phụ lục

1. **Demo:** Chương trình demo cho nội dung trong bài có thể được truy cập tại đây.
2. **Q&A:** Bạn có thể đặt thêm câu hỏi về nội dung bài đọc trong group Facebook hỏi đáp tại đây. Tất cả câu hỏi sẽ được trả lời trong vòng tối đa 4 tiếng.



Hình 10: Hình ảnh group facebook AIO Q&A.